

RC low-pass filter: frequency domain

A resistor and capacitor smooth a changing input. Take the output across the capacitor: slow changes pass through, while fast changes are attenuated.

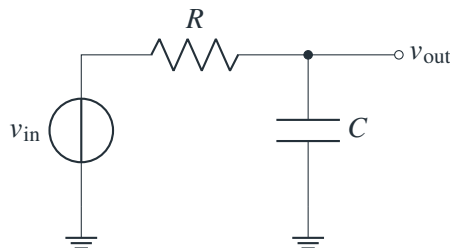
Assumption Ideal voltage source, ideal components, and no output load. For the step response, the capacitor is initially uncharged.

Transfer function and cutoff

$$H(s) = \frac{V_{\text{out}}(s)}{V_{\text{in}}(s)} = \frac{1}{1 + sRC}, \quad \tau = RC, \quad f_c = \frac{1}{2\pi RC}$$

Quantity	Meaning	Value
R	Series resistance	1.0 k Ω
C	Shunt capacitance	100 nF
τ	Time constant	100 μ s
f_c	Half-power cutoff frequency	1.59 kHz

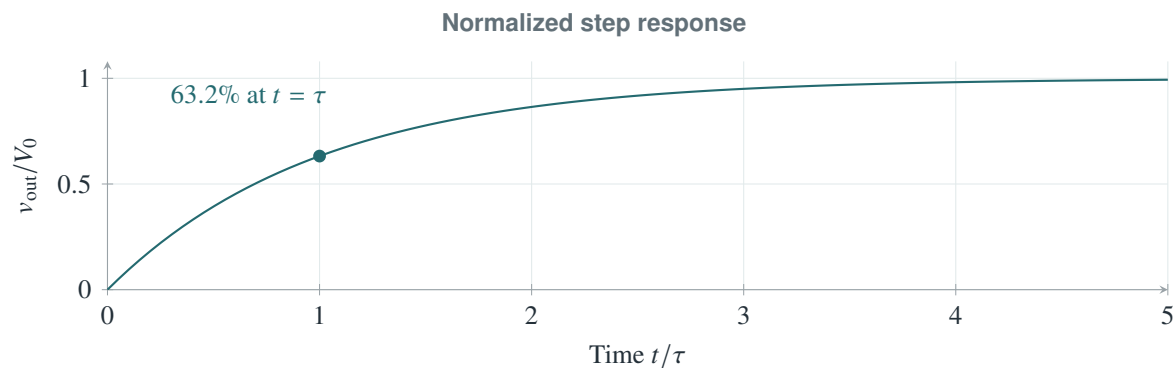
RC low-pass topology



Step response: time domain

Response to a step of amplitude V_0

$$v_{\text{out}}(t) = V_0(1 - e^{-t/\tau}), \quad t \geq 0$$



Frequency conventions. At DC, the gain tends to one. At the cutoff, $|H| = 1/\sqrt{2}$ and the phase is -45° . Use $\omega_c = 1/RC$ for rad/s and $f_c = 1/(2\pi RC)$ for Hz.

Sampled signals: definitions and notation

DEFINITION Sampling period

The interval T_s between successive samples. The sampling frequency is $f_s = 1/T_s$. A sampled input is written $x[n] = v_{\text{in}}(nT_s)$, where n is an integer sample index.

Symbol	Meaning	Unit
T_s	Sampling period	s
f_s	Sampling frequency	Hz
$x[n]$	Input voltage sample	V
$y[n]$	Filtered output sample	V
α	Discrete filter coefficient	dimensionless

From the circuit equation to a recurrence

Using a backward difference for the output derivative gives a simple discrete approximation of the continuous RC model, with $\tau = RC$.

Backward-Euler discretization

$$\begin{array}{ll}\tau \frac{dy}{dt} + y = x & \text{continuous model} \\ \tau \frac{y[n] - y[n-1]}{T_s} + y[n] = x[n] & \text{backward difference} \\ (\tau + T_s)y[n] = T_s x[n] + \tau y[n-1] & \text{collect terms} \\ y[n] = y[n-1] + \alpha(x[n] - y[n-1]) & \alpha = \frac{T_s}{\tau + T_s}\end{array}$$

Code fragments from the lecture

C: one sample update. Store the return value as the next state.

```
float lowpass_step(float x, float previous_y, float alpha)
{
    return previous_y + alpha * (x - previous_y);
}
```

ARM Thumb: set bit 5 in r1; r0 is a temporary register.

```
MOVS r0, #1      ; Start with a single set bit
LSLS r0, r0, #5   ; Move it to bit 5
ORRS r1, r1, r0   ; Set that bit in the flags word
```